

Plan de Adopción

Cortex · Edición 2026

*Cortex: Documentación como framework de gobernanza enterprise.
Harness controlado y trazable.*

AUTOR	Ezequiel Adrián Machuanin
FECHA	Mayo de 2026
LUGAR	Chaco, Argentina

Tu equipo ya usa inteligencia artificial. La pregunta no es si la adoptás. La pregunta es quién la gobierna.

1. Tesis y producto

El código dejó de escribirse mayoritariamente a mano: equipos chicos publican volúmenes de PRs que antes requerían oficinas enteras. Pero un porcentaje grande de ese código tiene problemas para llegar a producción — no porque los modelos sean malos, sino porque alrededor del modelo no hay nada: no hay memoria que sobreviva entre sesiones, no hay reglas que rijan qué se decide y lo peor de todo, es que no hay registro auditable de qué hizo cada agente.

La industria nombró esa capa que rodea al modelo: agent harness. `Agente = Modelo + Harness`. Los IDEs mas utilizados implementan harnesses personales (uno por sesión, uno por developer). Lo que ninguno ofrece es lo que aparece cuando ese developer deja de ser uno y pasa a ser un equipo: memoria persistente, governance declarativa, reglas uniformes cross-IDE.

Cortex es el framework de gobernanza que se conecta a los harnesses personales existentes — Claude Code, Cursor, Codex, Opencode y Pi — y les añade todo lo que un harness per-session personal no puede ofrecer.

2. Cómo está compuesto Cortex

Antes de entrar en mercado y estado actual conviene fijar el modelo mental. Cortex esta dividido en capas, y cada una resuelve uno de los huecos que un harness personal no cubre:

- **El vault** — el corazón. Un directorio (`vault/`) en tu repositorio con archivos planos en Markdown que contienen el conocimiento canónico del proyecto, organizado en 13 *doc types* formalizados (ADR, decision, spec, runbook, incident,

postmortem, etc.). Sin base de datos propietaria, sin servicio externo: si mañana te bajas de Cortex, el conocimiento ya está en tu repo y se queda con vos.

- **El servidor MCP** — el cerebro. Un proceso local que corre junto a tu IDE y expone ~30 herramientas al agente: cargar memoria histórica al abrir un ticket, indexar el vault, ejecutar verification-hooks, persistir sesiones, consultar relaciones del webgraph y demás herramientas.
- **Los comandos** `/cortex-*` — la superficie. Lo que el developer escribe con `/` en su IDE. Tres principales en cadena: `/cortex-sync` carga contexto al abrir trabajo, un middle pluggable ejecuta la tarea con ese contexto, y `/cortex-documenter` persiste decisiones y resultados al cerrar.
- **El vault enterprise** — la escala organizacional. Cuando se sale del proyecto, los vaults individuales se federan en un vault corporativo: el conocimiento crítico de negocio (decisiones de arquitectura recurrentes, criterios de compliance, runbooks operativos, lecciones aprendidas de incidents) atraviesa los proyectos. Cada agente que un developer arranca en cualquier proyecto **no parte de cero**, no actúa como un LLM genérico — arranca con el contexto de negocio acumulado de la organización y deja de ser un asistente de código para pasar a ser un actor de negocio más. Es la diferencia entre tener IA en el equipo y tener IA gobernada por el equipo.

3. Mercado, modelo y estado actual

Cliente ideal: Software factories y fintech pequeñas y medianas (PyMEs). El trigger de adopción es siempre el mismo: la industria entera detectó que cada developer usa la IA de forma distinta y los resultados son incoherentes o bien ya ocurrió algo no auditable que no se puede explicar.

Modelo de negocio: El framework es y va a seguir siendo gratuito al 100%. La monetización futura (2027 en adelante) viene por consultoría de implantación en empresas medianas/grandes: diseño del `org.yaml` corporativo, integración con stacks legacy, training, etc. Este camino permite construir comunidad y producto sin presión de monetización prematura.

Estado actual mediados de mayo de 2026 — la composición importa más que la magnitud:

FRENTE	ESTADO
Producto	MVP cierra en ~1 semana. Las tres capas operativas sobre los 5 IDEs validados (Claude Code, Cursor, Codex, Opencode, Pi); ~30 tools MCP expuestas; governance enforcement por timestamp activo; ONNX singleton estable.
Adopters	1 equipo de proyecto final confirmado usando Cortex como marco metodológico completo. 2 equipos adicionales acordados. 2 acercamientos industriales con reunión formal en ~1 semana (startup del sector + freelance senior que destinará un proyecto entero al uso intensivo).
Comunidad	Estrellas creciendo orgánicamente en GitHub; adopción individual para uso personal.

Los adopters confirmados están dispuestos a destinar proyectos enteros al uso intensivo de Cortex a cambio de feedback. Eso es exactamente el insumo que un framework joven necesita para madurar: usuarios comprometidos, no casuales.

4. Tres puestos, tres superficies

Cortex es un framework de gobernanza de documentación y conocimiento generado en la era de la IA. Las capacidades del framework se exponen distintas según el rol del usuario en la organización:

ROL	SUPERFICIE QUE USA	QUÉ OBTIENE
Developer	→ <code>/cortex-sync</code> → <code>/cortex-SDDwork</code> (pluggable) → <code>/cortex-documenter</code>	Memoria histórica del proyecto cargada automáticamente al abrir un ticket; propuesta interactiva antes de codear; documentación con criterio editorial al cierre; verification-hooks ejecutables que prueban objetivamente que el trabajo está hecho.
DevOps	→ Cortex CI plugin → Plantillas GitHub Actions / GitLab CI → Review sessions	Cada PR se valida contra la spec que lo originó (scope drift detection, hooks declarativos ejecutados). El servidor CI deja comentarios sticky con el resultado; review sessions persistidas con <code>mode=ci-review</code> para auditoría posterior.
Analista funcional	→ Cortex Webgraph → Snapshot navegable del vault → Memoria episódica vinculada por semántica y co-ocurrencia	Vista de grafo de cómo se conectan las decisiones, specs, ADRs y sessions del proyecto entero. Permite responder preguntas tipo " <i>¿cuáles fueron las decisiones que llevaron a este componente?</i> " sin tener que leer commits.

Los tres comparten la misma memoria subyacente — el vault canónico + la base episódica. Lo que cambia es la lente. Esa unificación es lo que hace que la información generada por developers sea inmediatamente útil para devops y analistas funcionales, sin trabajo manual de exportación.

5. La economía del contexto

Hay un argumento económico que justifica Cortex por sí solo, incluso en la versión gratuita: mejor contexto → menos errores → menos tokens consumidos a largo plazo.

Un agente sin memoria persistente redescubre el proyecto en cada sesión: re-lee los mismos archivos, repregunta las mismas decisiones y repite errores ya cometidos al lado. Cada redescubrimiento cuesta tokens. A escala de equipo, esos tokens son una línea visible en la factura mensual de IA.

Cortex ataca esa ineficiencia de tres formas:

1. **Recuperación dirigida con Reciprocal Rank Fusion** — el agente es inyectado con contexto histórico relevante al abrir el ticket (`cortex_sync_ticket`), no exploración a ciegas. Menos lecturas redundantes, menos tokens.
2. **Memoria episódica + semántica vectorizadas** — embeddings ONNX locales (sin costo de API), índice persistente, recuperación por similitud en milisegundos.
3. **Datos históricos conectados** — el webgraph relaciona decisiones, specs y resultados. Una decisión de arquitectura tomada en el Proyecto A se vuelve consultable como evidencia al iniciar el Proyecto B, sin necesidad de re-explorar el problema desde cero.

A esto se le suma algo indispensable: La trazabilidad. Cada decisión queda persistida en su `doc_type` canónico, vectorizada y conectada. Nada se reconstruye sin pensar en el futuro.

6. Roadmap

HORIZONTE	HITO PRINCIPAL	CÓMO SE MIDE
~1 semana hoy → fin de mayo 2026	Cierre de MVP. Modelo triádico completo, los 5 IDEs validados en proyectos reales, los 13 doc types operativos cross-IDE, ONNX singleton estable.	El MVP cierra cuando los +3 equipos que estan desarrollando su proyecto final puedan instalar Cortex en su proyecto y utilizarlo de forma asistida, brindando feedback constante.
3 meses jun – ago 2026	Consolidar y validar el MVP con adopters de confianza.	Cortex instalado en las dos empresas ya en conversación (startup del sector + freelance senior), y agregar un par más del mismo tamaño y nivel de confianza. El objetivo no es escalar sino estabilizar el framework con feedback cercano.
3 a 6 meses sep 2026 – feb 2027	Primera ampliación a empresas con métodos consolidados.	Implementación en empresas con equipos más consolidados, procesos internos definidos y posible historial técnico previo, para observar cómo el framework responde ante estructuras menos flexibles que las de las pymes iniciales.

7. Qué busco

Para crecer, Cortex necesita tres cosas concretas a mediano plazo:

1. **Adopters serios** — equipos técnicos dispuestos a probar Cortex en un proyecto real (no sandbox) y dar feedback honesto. A cambio: acceso gratuito perpetuo, influencia directa en el roadmap, reconocimiento público como adopter fundacional, soporte directo del founder.
2. **Inversores ángel estratégicos** (no financieros) — personas con red de contactos y experiencia en go-to-market de developer tools, no capital.

No busco capital de riesgo en esta etapa. Una ronda hoy presionaría al producto a optimizar por crecimiento antes de que la base técnica esté estabilizada, y el

framework necesita justamente lo contrario: tiempo para asentarse con adopters cercanos antes de escalar.

CONTACTO	Ezequiel Adrián Machuanin ezequieladrianmachuanin@gmail.com
EMAIL	ezequieladrianmachuanin@gmail.com
REPOSITORIO	github.com/MachuaninEzequiel/Cortex
